Exp 3 : To write a program to demonstrate process creating (Parent and Child).

a) fork() function : 1

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gedit cprocess_1290.c


^C
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ cat cprocess_1290.c
#include<stdio.h>
#include<unistd.h>

int main()
{
    int forkid;
    forkid = fork();

    if(forkid < 0)
    {
        printf("Fork failed to create process\n");
    }
    else if(forkid == 0)
    {
        printf("Process created\n");
        printf("Process id of the child process %d\n", getpid());
        printf("Process id of the parent class %d\n", getppid());
    }
    else
    {
        printf("In parent process\n");
    }

    return 0;
}
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc cprocess_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
In parent process
Process created
Process id of the child process 6339
Process id of the parent class 1888
```

b) fork() function : 2

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gedit cp2_1290.c
^C
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ cat cp2_1290.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int forkid1, forkid2;

    forkid1 = fork();  // First fork

    if (forkid1 < 0) {
        printf("First fork failed\n");
    }
    else if (forkid1 == 0) {
        // Child process 1
        printf("This is child process 1 with PID %d, parent PID %d\n", getpid(), getppid());
    }
    else {
        // Parent process continues and forks again
        forkid2 = fork();  // Second fork

        if (forkid2 < 0) {
            printf("Second fork failed\n");
        }
        else if (forkid2 == 0) {
            // Child process 2
            printf("This is child process 2 with PID %d, parent PID %d\n", getpid(), getppid());
        }
        else {
            // Original parent process
            printf("This is the parent process with PID %d\n", getpid());
        }
    }

    return 0;
}
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc cp2_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
This is child process 1 with PID 6620, parent PID 6619
This is the parent process with PID 6619
This is child process 2 with PID 6621, parent PID 6619
```

c) Fork() function : 3

```
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gedit cp3_1290.c
^C
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ cat cp3_1290.c
#include <stdio.h>
#include <unistd.h>

int main() {
    int forkid1, forkid2, forkid3;

    forkid1 = fork();

    if (forkid1 < 0) {
        printf("First fork failed\n");
    }
    else if (forkid1 == 0) {
        // Child process 1
        printf("Child process 1 PID %d, parent PID %d\n", getpid(), getppid());
    }
    else {
        forkid2 = fork();

        if (forkid2 < 0) {
            printf("Second fork failed\n");
        }
        else if (forkid2 == 0) {
            // Child process 2
            printf("Child process 2 PID %d, parent PID %d\n", getpid(), getppid());
        }
        else {
            forkid3 = fork();

            if (forkid3 < 0) {
                printf("Third fork failed\n");
            }
            else if (forkid3 == 0) {
                // Child process 3
                printf("Child process 3 PID %d, parent PID %d\n", getpid(), getppid());
            }
            else {
                // Parent process
                printf("Parent process PID %d\n", getpid());
            }
        }
    }

    return 0;
}
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ gcc cp3_1290.c
lab-03-17@lab-03-17-OptiPlex-3280-AIO:~$ ./a.out
Child process 1 PID 6788, parent PID 6787
Parent process PID 6787
Child process 2 PID 6789, parent PID 6787
Child process 3 PID 6790, parent PID 6787
```